

Problem Statement & Instructions

Read the problem statement, create a working solution and write supporting tests. The solution should be written in one of the following languages: java, python, go, C# (.Net Core on Linux) or scala. Be prepared to explain your solution during a review.

Throughout this document, we use terms defined in [RFC-2119](#); you MUST follow these definitions when completing this assignment.

The Problem

We want you to create a production-ready, command-line application that will calculate the league standing table for a football (soccer) league. Then use this code to calculate league standings for the English First Division in the 10th week of the 1974/75 season. You do not need to be an expert in sports or football to complete this assignment, but you may need to do some research.

The rules

The rules for calculating league standings should follow those used by the English First Division in the season in question.

Input/output

The input and output SHOULD be text csv files. Either using stdin/stdout or taking filenames on the command. You MUST include these files in your submission.

The input SHOULD contain the results of games, one per line.

The output should be a league standing table in the conventional format.

Guidelines

If you use libraries installed by a common package manager (e.g pip), you SHOULD NOT commit the installed packages.

At SPAN we create automated tests you MUST include them in your submission.

If any complicated setup steps are required to run your solution, please document them.

Platform

This will be run in a unix-ish environment (OS X). If you choose to use a compiled language, please keep this in mind. Please use platform-agnostic constructs where possible.

AI Collaboration

We expect you to use AI coding assistants (such as Claude Code) during this exercise. Your ability to effectively collaborate with AI tools is a significant part of what we're evaluating.

In addition to your working solution, submit the following AI collaboration artifacts via your Git repository (**the below example uses Claude Code, if you use some other AI you must submit equivalent artifacts**). These artifacts are mandatory. Submissions missing any of these artifacts are incomplete and will not proceed past the coding test phase.

Project configuration / instructions

- Include your `CLAUDE.md` file in your project root

AI session data

- Include the `.claude/` folder from your project root

Conversation history

- Copy the contents of `~/.claude/projects/<project-path-and-name>/` into an `./ai/` folder in your project

AI reflection

- Include an `AI_REFLECTION.md` in your project root. In a few paragraphs, describe a moment where the AI took a wrong direction or produced something you disagreed with, and explain how you responded. If nothing went wrong, describe a decision you made that shaped the solution in a meaningful way.

Your submission should include:

None

```
your-project/
├── README.md           # Setup and run instructions
├── CLAUDE.md           # Project instructions for AI
└── AI_REFLECTION.md    # Your reflection on the AI collaboration
```

```
|— .claude/           # Claude session data
|— ai/               # Conversation history exports
|— ....             # Rest of your solution
```
